

Weekend Homework — Python Fundamentals

RiverEdge Analytics Internship · Total time: ~4 hours · Due: Monday standup

Rules

1. One .py file per problem (p01.py, p02.py ...), committed to your Git repo as you finish each — not all at once on Sunday night.
2. NO AI tools. These problems are written specifically for this office — pasted answers are easy to spot, and on Monday you will explain any 2 lines I pick. Code you cannot explain = whole homework redone.
3. Stuck? Apply the 30-minute rule, then write a comment in the file describing what you tried. An honest "stuck here" earns more respect than fake code.
4. Every program must run without errors even on bad input.

Part A — Warm-up (about 45 minutes)

Problem 1: Name Badge Generator

Estimated time: ~15 min

Ask the user for their full name and the company name. Print an office badge exactly in this format: initials in capitals, followed by the surname, then the badge width padded with = signs so every badge is exactly 30 characters wide.

```
Input: name = "balai divyesh devshibhai", company = "RiverEdge"
Output:
=====
| B.D. DEVSHIBHAI          |
| RiverEdge Intern         |
=====
```

Problem 2: Chai Break Splitter

Estimated time: ~15 min

The office chai order costs a total amount (input 1) shared by N people (input 2). Each person pays a whole-rupee share; the leftover paise go into the office piggy bank. Print each share and the piggy bank amount. Handle the case where N is 0 with a clean message instead of a crash.

```
Input: total = 173, people = 4
Output: Each pays: Rs 43 | Piggy bank gets: Rs 1
```

Problem 3: Password Strictness Meter

Estimated time: ~15 min

Read a password and score it: +1 if length is 8 or more, +1 if it has both upper and lower case, +1 if it has a digit, +1 if it has any of @ # \$ %. Print the score and the verdict: 0-1 Weak, 2-3 Okay, 4 Strong. Do not use any imports.

```
Input: "RiverEdge#25"
Output: Score: 4/4 - Strong
```

Part B — Core practice (about 2 hours)

Problem 4: Attendance Cleaner

Estimated time: ~20 min

The biometric machine exports names messily: extra spaces, random capitals, and duplicates when someone punches twice. Given the list below (hardcode it), produce a clean, alphabetically sorted list of unique proper-case names and print how many duplicate punches were removed.

```
raw = [' divyesh BALAI', 'rajat ', 'Divyesh Balai', 'RAJAT', 'khushi shah', ' Khushi Shah']
Output:
Clean list: ['Divyesh Balai', 'Khushi Shah', 'Rajat']
Duplicates removed: 3
```

Problem 5: Meeting Room Clash Detector

Estimated time: ~25 min

Bookings are tuples of (room, start_hour, end_hour) in 24h format. Hardcode the list below. Print every pair of bookings that clash (same room, overlapping hours). If there are no clashes print "Schedule OK".

```
bookings = [('Falcon', 10, 11), ('Falcon', 10.5, 12), ('Tiger', 10, 11), ('Falcon', 14, 15), ('Tiger', 11, 12)]
Output: CLASH in Falcon: (10, 11) overlaps (10.5, 12)
```

Problem 6: Salary Slip Word-o-meter

Estimated time: ~25 min

HR wants amounts in words on salary slips. Write a function `amount_in_words(n)` for `n` from 0 to 9999 that returns the amount in Indian style words. Build it yourself with dictionaries — no libraries.

```
amount_in_words(4250) -> "four thousand two hundred fifty rupees"
amount_in_words(807)  -> "eight hundred seven rupees"
```

Problem 7: The Forgetful Intern Log

Estimated time: ~25 min

An intern logs work as one string: task codes separated by commas, but sometimes types spaces or repeats a code, and sometimes the codes are in mixed case. Write a function that takes the log string and returns a dictionary of code -> count (codes uppercased), ignoring empty entries. Then print the most frequent task.

```
log = "py, PY, excel, py , , SQL, excel, PY"
Output: {'PY': 4, 'EXCEL': 2, 'SQL': 1} | Most frequent: PY
```

Problem 8: Odd One Out in the Invoice

Estimated time: ~25 min

Every valid invoice number in this office follows the pattern: 2 letters + 4 digits (e.g., RE2026). Given a hardcoded list of invoice numbers, print the invalid ones and WHY each is invalid (wrong length / letters part wrong / digits part wrong). No regex allowed — use string methods and slicing.

```
invoices = ['RE2026', 'R12026', 'REX206', 'RE20A6', 'RE12345']
Output:
R12026 -> letters part wrong
REX206  -> digits part wrong
RE20A6  -> digits part wrong
RE12345 -> wrong length
```

Part C — Challenge (about 1 hour 15 minutes)

Problem 9: Mini Chai Ledger (file handling)

Estimated time: ~35 min

Create a program with a menu that loops: 1) Add expense (name + amount), 2) Show all expenses, 3) Show total per person, 4) Exit. Expenses must be SAVED to a file `chai_ledger.txt` (one line per expense, e.g. "Rajat,15") and loaded back when the program restarts — closing the program must not lose data. Wrap all file reading in `try/except` so a missing or corrupted file never crashes the menu.

```
Menu:
1. Add  2. Show all  3. Totals  4. Exit
> 3
Rajat: Rs 45
Divyesh: Rs 30
```

Problem 10: Sensor Zigzag

Estimated time: ~40 min

A temperature sensor gives one reading per hour (hardcode any list of 10-12 numbers). Find and print: (a) the longest streak of strictly rising readings, (b) the hour at which the biggest single jump happened, and (c) a text graph — one line per reading made of * characters (1 star per 2 degrees, rounded down). Everything from scratch: no `max()`, no `min()` anywhere in the file.

```
readings = [18, 21, 25, 24, 26, 30, 29, 31, 35, 34]
(a) Longest rising streak: 18->21->25 (length 3)
(b) Biggest jump: hour 8 (31 to 35)
(c) 18 | *****
```

21 | *****
...

Bonus (optional, no time limit)

Problem 11: FizzBuzz, But Make It Ours

Estimated time: open

Print numbers 1 to 60. Multiples of 4 print "Chai", multiples of 6 print "Break", multiples of both print "ChaiBreak" — but with a twist: if the number also contains the digit 3 anywhere, the word must be printed reversed. Write it once with if/else, then a second version in at most 3 lines.

```
12 -> ChaiBreak    36 -> kaerBiahC (multiple of both AND contains 3)
```

Problem 12: Explain-Back Prep

Estimated time: 15 min, mandatory

For any TWO problems you found hardest, write 3-4 sentences at the bottom of the file as comments: what your approach is, and one thing you tried that did NOT work. This is what you will present at Monday standup.

Marking (out of 20): 12 for working code (1 per problem, 2 for P9 and P10) · 4 for clean naming and comments · 2 for steady Git commits over the weekend · 2 for the Monday explain-back. Copying from each other: both submissions get zero — your solutions to these problems will not look alike by accident.